Estimating Synaptic Connectivity from Neural Activity

A Linear-Regression Approach — An Educational Chapter

Mohammad Joudy
Bernstein Center Freiburg

June 22, 2026

Abstract

This chapter explains, from first principles, how the question *"which neurons are connected to which?"* can be turned into a *linear regression*. We start from one simple physical idea — a neuron's activity is shaped by the recent activity of the other neurons — and build the regression up one step at a time, from a single neuron to the whole network. We then show where this idea comes from (the underlying linear dynamics of neural networks), how the regression is actually solved, and how three things change the picture: the *external input*, the meaning of the *residual*, and the use of *regularization*. No specialist background is assumed.

Contents

1 The question and the data

We record the activity of N neurons over time and want to recover their **connectivity**: who connects to whom, whether each connection is *excitatory* (one neuron makes another more active) or *inhibitory* (less active), and how strong it is.

The data. The activity is measured with two-photon calcium imaging. Calcium fluorescence is a slow, blurred echo of a neuron's spikes. A fixed **preprocessing** step (deconvolution) turns each fluorescence trace back into a per-neuron *activity feed*. From here on,

$$x_i(t) \;=\; \text{activity (the preprocessed feed) of neuron } i \text{ at time } t.$$

Preprocessing is part of the method and is not revisited in this chapter — the feed $x_i(t)$ is simply our starting signal.

How we test it. To know whether an estimate is right, we use *simulated* networks (of the Brunel type) where the true connectivity is known in advance. We run the method on the simulated activity and compare the estimate to the ground truth.

2 Where the regression comes from: the underlying theory

Before turning the problem into a regression, it helps to see the physical model the regression is built on.

The idea. A neuron's activity in the next instant is driven by the activity of the neurons that connect to it. Written as a continuous-time law for the whole network:

$$\frac{d\mathbf{x}}{dt} \;=\; G\,\mathbf{x}(t) \;+\; \mathbf{u}(t), \tag{1}$$

where $\mathbf{x}(t)$ is the vector of all N activities, $\mathbf{u}(t)$ is the external input, and G is the $N \times N$ *coupling matrix*. The off-diagonal entry G_{ij} says how strongly neuron j drives neuron i.[1]

Why a *linear* law is reasonable. Real neurons are nonlinear (they fire all-or-none spikes). Equation (1) is a *first-order approximation* that is accurate when the network sits in a stable, irregular operating state — the *asynchronous-irregular* regime typical of cortex. Near such an operating point, small fluctuations of activity really do add up approximately linearly. This is the same "linear response" reasoning used throughout physics, and it is the bridge from the work of Rotter & Diesmann (1999) to our setting. The coupling then factorizes as

$$G_{ij} \;=\; \frac{\text{gain}_i \cdot W_{ij}}{\tau} \qquad (i \neq j), \tag{2}$$

where W_{ij} is the true *synaptic* weight, gain_i is how strongly neuron i converts input into output, and τ is a time constant. The important message of (2) is: G **is not the synaptic matrix** W **itself, but a gain-scaled version of it.**

[1] In the original literature the state is often called y and the input x; here we call the activity x and the input u, so that x_i always means "neuron i".

From the law to one time step. Solving (1) over one small step Δ gives an exact update rule:

$$\mathbf{x}(t) \;=\; \underbrace{\exp(G\,\Delta)}_{\mathbf{A}}\;\mathbf{x}(t-\Delta) \;+\; \boldsymbol{\varepsilon}(t), \tag{3}$$

with $\boldsymbol{\varepsilon}(t)$ collecting the input and noise over that step. The single matrix

$$\boxed{\mathbf{A} \;=\; \exp(G\,\Delta)}$$

is called the *one-step propagator*. **This matrix A is exactly what the regression will estimate.**

Why A carries the connectivity. For a small step, $\exp(G\Delta) \approx I + G\Delta$, so for $i \neq j$

$$A_{ij} \;\approx\; \Delta\, G_{ij} \;\propto\; \text{gain}_i \cdot W_{ij}.$$

Hence the *sign* of A_{ij} tells excitatory from inhibitory, and its *size* grows with the synaptic weight. Reading connectivity off $\mathbf{A}$ is therefore meaningful — with the caveat that what we read is *effective* connectivity (gain-scaled), not the raw synaptic weight.

3 From the model to a regression, step by step

We now rebuild Eq. (3) from the ground up, the way one would teach it. We use the simplest version: each neuron at time t depends on all neurons at time $t-1$ (a one-step lag).

3.1 Step 1 — one neuron

Take neuron 1. Its activity at time t is a weighted sum of *every* neuron's activity at $t-1$:

$$x_1(t) = a_{11}x_1(t{-}1) + a_{12}x_2(t{-}1) + a_{13}x_3(t{-}1) + \cdots + a_{1N}x_N(t{-}1) + \varepsilon_1(t). \tag{4}$$

Here a_{12} is how strongly neuron 2 drives neuron 1, a_{13} how strongly neuron 3 drives it, and so on. The last term $\varepsilon_1(t)$ is the *leftover* (residual): everything not explained by the network — external input plus noise. Each a_{1j} is an unknown number we want to find.

3.2 Step 2 — every neuron

Write the same line for neurons $1, 2, 3, \ldots, N$:

$$\begin{aligned}
x_1(t) &= a_{11}x_1(t{-}1) + a_{12}x_2(t{-}1) + \cdots + a_{1N}x_N(t{-}1) + \varepsilon_1(t),\\
x_2(t) &= a_{21}x_1(t{-}1) + a_{22}x_2(t{-}1) + \cdots + a_{2N}x_N(t{-}1) + \varepsilon_2(t),\\
x_3(t) &= a_{31}x_1(t{-}1) + a_{32}x_2(t{-}1) + \cdots + a_{3N}x_N(t{-}1) + \varepsilon_3(t),\\
&\;\;\vdots\\
x_N(t) &= a_{N1}x_1(t{-}1) + a_{N2}x_2(t{-}1) + \cdots + a_{NN}x_N(t{-}1) + \varepsilon_N(t).
\end{aligned} \tag{5}$$

That is N equations, one per neuron. All the unknown weights a_{ij} form a table — the matrix $\mathbf{A}$:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1N} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2N} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3N} \\ \vdots & & & \ddots & \vdots \\ a_{N1} & a_{N2} & a_{N3} & \cdots & a_{NN} \end{bmatrix}. \tag{6}$$

Row i, column j is a_{ij} = the connection *from* neuron j *to* neuron i. This table is the connectivity we are after.

3.3 Step 3 — stack the neurons into a snapshot

Collect the N activities at one time into a single column (a "snapshot"):

$$\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ \vdots \\ x_N(t) \end{bmatrix}.$$

Then the whole block (5) shrinks to one line:

$$\mathbf{x}(t) \;=\; \mathbf{A}\,\mathbf{x}(t{-}1) \;+\; \boldsymbol{\varepsilon}(t). \tag{7}$$

In words: *this snapshot = table* $\mathbf{A}$ *applied to the previous snapshot, plus leftovers.* This is exactly Eq. (3) from the theory.

3.4 Step 4 — use many time steps

A single pair of snapshots cannot pin down $N \times N$ unknowns. But a recording gives thousands of consecutive pairs, and every pair obeys the *same* table $\mathbf{A}$:

$$\begin{aligned} \mathbf{x}(2) &= \mathbf{A}\,\mathbf{x}(1) + \boldsymbol{\varepsilon}(2), \\ \mathbf{x}(3) &= \mathbf{A}\,\mathbf{x}(2) + \boldsymbol{\varepsilon}(3), \\ \mathbf{x}(4) &= \mathbf{A}\,\mathbf{x}(3) + \boldsymbol{\varepsilon}(4), \\ &\vdots \\ \mathbf{x}(T) &= \mathbf{A}\,\mathbf{x}(T{-}1) + \boldsymbol{\varepsilon}(T). \end{aligned}$$

Line up the "previous" snapshots as columns of one matrix and the "current" ones as columns of another:

$$\mathbf{X}_{\text{past}} = \big[\,\mathbf{x}(1)\;\mathbf{x}(2)\;\cdots\;\mathbf{x}(T{-}1)\,\big], \qquad \mathbf{X}_{\text{next}} = \big[\,\mathbf{x}(2)\;\mathbf{x}(3)\;\cdots\;\mathbf{x}(T)\,\big].$$

All time steps at once become a single matrix equation:

$$\boxed{\mathbf{X}_{\text{next}} \;=\; \mathbf{A}\,\mathbf{X}_{\text{past}} \;+\; \mathbf{E}} \tag{8}$$

This is a **multivariate linear regression**: predict $\mathbf{X}_{\text{next}}$ from $\mathbf{X}_{\text{past}}$ with the unknown weight table $\mathbf{A}$.

4 Solving the regression

4.1 The objective

We cannot make the leftovers $\mathbf{E}$ exactly zero (there is always input and noise). Instead we choose the $\mathbf{A}$ whose prediction $\mathbf{A}\mathbf{X}_{\text{past}}$ is as close as possible to $\mathbf{X}_{\text{next}}$ — smallest total squared error:

$$\widehat{\mathbf{A}} \;=\; \arg\min_{\mathbf{A}} \;\big\|\,\mathbf{X}_{\text{next}} - \mathbf{A}\,\mathbf{X}_{\text{past}}\,\big\|_F^2, \tag{9}$$

where $\|\cdot\|_F^2$ just means "sum of all squared entries".

4.2 The closed-form solution

Setting the derivative of (9) with respect to $\mathbf{A}$ to zero gives the *normal equations*, and solving them yields:

$$\boxed{\widehat{\mathbf{A}} \;=\; \mathbf{C}_{\mathrm{np}}\,\mathbf{C}_{\mathrm{pp}}^{-1}} \qquad \begin{aligned} \mathbf{C}_{\mathrm{pp}} &= \mathbf{X}_{\mathrm{past}}\,\mathbf{X}_{\mathrm{past}}^{\top} \quad \text{(past vs. past)},\\ \mathbf{C}_{\mathrm{np}} &= \mathbf{X}_{\mathrm{next}}\,\mathbf{X}_{\mathrm{past}}^{\top} \quad \text{(next vs. past)}. \end{aligned} \tag{10}$$

Both $\mathbf{C}_{\mathrm{pp}}$ and $\mathbf{C}_{\mathrm{np}}$ are only $N \times N$, *no matter how long the recording is*. They are sums over time steps, so they can be accumulated chunk by chunk — which is why the method scales to very long recordings and large N without ever loading all the data at once.

4.3 When there is no formula

The clean formula (10) exists only for plain least squares and for the ridge penalty (Section 6). Once we add a sparsity penalty, no closed form exists, and we instead *iterate*: start from a guess, repeatedly nudge $\mathbf{A}$ downhill on the objective (gradient descent / FISTA) until it stops improving. The target is the same; only the route to it changes.

5 The external input and the residual

The leftover term $\boldsymbol{\varepsilon}(t)$ deserves its own section, because it decides whether the estimate is trustworthy.

What the residual is. At the true $\mathbf{A}$, the leftover is

$$\boldsymbol{\varepsilon}(t) \;=\; \mathbf{x}(t) - \mathbf{A}\,\mathbf{x}(t-1) \;=\; \underbrace{\mathbf{u}(t)}_{\text{external input}} + \text{noise}. \tag{11}$$

So the residual is *the external input plus noise*. There are two ways to handle the input, and they differ only in *where the input is put*.

Version 1 — input left in the residual (what the method does). We subtract each neuron's mean ("centering") and do not model the input explicitly. The constant part of the input sets the average activity and is removed by centering; the fluctuating part stays inside $\boldsymbol{\varepsilon}(t)$.

Version 2 — input modeled explicitly. We keep $\mathbf{u}(t)$ as a separate term, so the residual contains only noise.

When are the two the same? Least squares gives the correct (unbiased) $\mathbf{A}$ *exactly when the residual is uncorrelated with the regressor* $\mathbf{x}(t-1)$. Everything hinges on one question: *is the external input correlated with the network's own past activity?*

- **Case A — independent input** (e.g. a separate, random drive to each neuron, as in the Brunel simulation). The input is uncorrelated with the past, so Versions 1 and 2 give the *same* $\mathbf{A}$. The input only *inflates the residual variance* — you need more data for the same precision, but the estimate is **not biased**.
- **Case B — shared / common input** (the same external signal drives several neurons at once). Now the input *is* correlated with the past, the residual is correlated with the regressor, and $\mathbf{A}$ becomes **biased**: it reports *phantom connections* between neurons that merely share input but have no synapse. This is the same thing as the "hidden neuron" problem: an unobserved neuron driving two visible ones looks like a connection between them.

How incomplete is the theory if we ignore the input? Under independent input (our simulations) it is *not* incomplete for getting connectivity right — ignoring the input costs only precision, not correctness. Under common input (likely in real brains) it is genuinely incomplete: the bias is a *confound*, and no amount of regularization removes a confound. This is the single most important limitation to state honestly.

A free diagnostic. Because the residual *is* the input plus noise, its structure is informative. Compute the residuals and check their correlation across neurons: *uncorrelated* residuals indicate Case A (safe); *correlated* residuals warn of common input (Case B).

6 Regularization: different versions

Plain least squares can overfit: with N^2 unknowns and limited, noisy data it tends to fill the matrix with small spurious entries. *Regularization* adds a penalty that encodes what we believe about connectivity — mainly that it is *sparse* (most pairs are not connected). Each penalty is a different hypothesis.

6.1 Ridge (L2): shrink everything

$$\min_{\mathbf{A}} \ \|\mathbf{X}_{\text{next}} - \mathbf{A}\mathbf{X}_{\text{past}}\|_F^2 + \lambda\|\mathbf{A}\|_F^2 \quad \Longrightarrow \quad \widehat{\mathbf{A}} = \mathbf{C}_{\text{np}}\,(\mathbf{C}_{\text{pp}} + \lambda I)^{-1}. \tag{12}$$

Adding λI stabilizes the inverse and pulls all entries toward zero, but *none* become exactly zero — the matrix stays dense. Still a closed form.

6.2 Lasso (L1): force entries to exactly zero

$$\min_{\mathbf{A}} \ \|\mathbf{X}_{\text{next}} - \mathbf{A}\mathbf{X}_{\text{past}}\|_F^2 + \lambda\|\mathbf{A}\|_1, \qquad \|\mathbf{A}\|_1 = \sum_{ij} |a_{ij}|. \tag{13}$$

The L1 penalty sets small weights *exactly* to zero, producing a sparse matrix — directly addressing the spurious connections. It has no closed form (the absolute value is not differentiable at zero), so it is solved iteratively (FISTA / coordinate descent).

6.3 Elastic Net: sparse *and* stable

$$\min_{\mathbf{A}} \ \|\mathbf{X}_{\text{next}} - \mathbf{A}\mathbf{X}_{\text{past}}\|_F^2 + \lambda_1\|\mathbf{A}\|_1 + \lambda_2\|\mathbf{A}\|_F^2. \tag{14}$$

Elastic Net combines the two penalties: L1 *selects* (drives most entries to exactly zero) and L2 *stabilizes*.

Why not Lasso (L1) alone? Lasso's well-known weakness is *correlated predictors*. When two neurons have nearly identical activity the fit cannot tell them apart, so Lasso keeps one and zeros the other *arbitrarily* — and which one it keeps can flip with a little extra noise or data. A genuine connection can therefore be dropped only because a correlated neighbour "won the coin toss". Adding the L2 term cures this through the *grouping effect*: rather than picking a single winner, L2 shares similar weight among correlated neurons, so the selection is stable and the underlying linear algebra is better conditioned. Elastic Net thus keeps Lasso's sparsity while removing its fragility, which is why it is the safer default.

A caveat for the present data. The benefit of L2 is only as large as the correlations it guards against. In the $N = 100$ network at the optimal lag the neurons are weakly correlated, so Lasso and Elastic Net score almost the same and L2 is nearly idle. The reason to keep it is forward-looking: as networks grow larger and denser the inter-neuron correlations increase and Lasso's instability begins to bite — exactly the regime Elastic Net is insurance against.

6.4 Biological constraints (Dale's law)

A real neuron is typically *either* excitatory *or* inhibitory for all of its outgoing connections (Dale's law). Let $t_j \in \{+1, -1\}$ be the type of the *source* neuron j ($+1$ excitatory, -1 inhibitory); Dale's law says every entry in column j of $\mathbf{A}$ should share that sign. We can write this as a penalty added to the Elastic Net loss:

$$L_{\text{Dale}}(\mathbf{A}) = \|\mathbf{X}_{\text{next}} - \mathbf{A}\mathbf{X}_{\text{past}}\|_F^2 + \lambda_1\|\mathbf{A}\|_1 + \lambda_2\|\mathbf{A}\|_F^2 + \lambda_D \sum_{i \neq j} \big[-t_j\, a_{ij} \big]_+, \tag{15}$$

where $[z]_+ = \max(z, 0)$. The final term — a one-sided *hinge* — is zero when an entry has the correct sign ($t_j a_{ij} \geq 0$) and grows with the size of the violation when the sign is wrong; λ_D sets how dearly a wrong sign is charged. Letting $\lambda_D \to \infty$ forbids wrong signs outright, giving the equivalent *hard-constrained* form

$$\widehat{\mathbf{A}} = \arg\min_{\mathbf{A}} \ \|\mathbf{X}_{\text{next}} - \mathbf{A}\mathbf{X}_{\text{past}}\|_F^2 + \lambda_1\|\mathbf{A}\|_1 + \lambda_2\|\mathbf{A}\|_F^2 \quad \text{s.t.} \quad t_j\, a_{ij} \geq 0 \ \ \forall\, i \neq j. \tag{16}$$

How it is solved (and why *during* the fit matters). The hard form (16) is enforced *inside* the iterative solver: after each gradient-plus-soft-threshold step, every column is projected onto its allowed sign (keep correct-sign entries, zero the wrong-sign ones) — the $\lambda_D \to \infty$ limit applied directly. Doing this during the fit, rather than as a one-off cleanup at the end, is what makes it work: each time a wrong-sign entry is zeroed, the next gradient step lets the surviving correct-sign entries *re-grow* to explain that variance. A post-hoc cleanup instead deletes wrong-sign entries once and discards whatever they were explaining — which is why the in-solver version detects connections better. The source types t_j are taken from an initial Elastic-Net fit (the sign of each neuron's strongest outgoing entry); the soft form (15) with finite λ_D is the natural "soft Dale" variant to sweep.

Method	**Penalty**	**Sparse?**	**Closed form?**	**Solver**
Least squares (OLS)	none	no	yes	normal equations
Ridge	L2	no	yes	normal equations $+\lambda I$
Lasso	L1	**yes**	no	iterative (FISTA)
Elastic Net	L1+L2	**yes**	no	iterative (FISTA)
Dale-constrained	sign	**yes**	no	projected iterative

Table 1: The regularization variants, all minimizing the same reconstruction error with different penalties.

7 Reading and interpreting the result

For each entry of the solved table $\widehat{\mathbf{A}}$:

$$a_{ij} > 0 \Rightarrow j \text{ excites } i, \qquad a_{ij} < 0 \Rightarrow j \text{ inhibits } i, \qquad a_{ij} \approx 0 \Rightarrow \text{no connection.}$$

The diagonal a_{ii} is the neuron's own carry-over (its intrinsic time constant), not a connection, so it is discarded.

Recall from Section 2 that $a_{ij} \propto \text{gain}_i \cdot W_{ij}$. So the estimate gives the right *sign* and *relative* strength of connections — this is the *effective* connectivity. Recovering the raw synaptic weight W_{ij} requires dividing out the per-neuron gain, which is what rescaling / balance corrections aim to do. When ground truth is available, the quality is scored by how well $\widehat{\mathbf{A}}$ matches the true matrix (e.g. correlation, or area under the ROC curve for detecting connections).

8 The whole picture, and the assumptions

The cascade. Reading left to right is how the data is generated; the inference runs it backwards:

$$W \;\to\; G \;\to\; \mathbf{A} = \exp(G\Delta) \;\to\; \text{activity} \;\to\; \text{calcium}$$
$$\xrightarrow{\text{deconvolve}} \text{feed } \mathbf{x} \xrightarrow{\text{regression}} \widehat{\mathbf{A}} \;\to\; \text{connectivity.}$$

The assumptions, in one place.

1. The network is approximately linear (valid in the asynchronous-irregular regime).
2. One time-lag is enough (the step Δ is short).
3. Calcium is a linear filter of activity, with a known/estimated time constant.
4. Deconvolution approximately inverts that filter (similar time constant across neurons).
5. The residual is uncorrelated with the past — i.e. *independent* external input (Section 5).
6. Enough data: the number of time steps T greatly exceeds N^2.

Assumption 5 is the one most likely to fail in real data, and it is exactly the external-input / common-input issue. Everything else is either guaranteed by the simulation or testable by a parameter sweep.

Summary

> "Each neuron depends on all the others" → one equation per neuron → stack neurons into a snapshot → stack many snapshots over time → find the one table $\mathbf{A}$ that predicts best → that table *is* the connectivity.

The least-squares solution $\widehat{\mathbf{A}} = \mathbf{C}_{\text{np}}\mathbf{C}_{\text{pp}}^{-1}$ is the core; the external input lives in the residual and is harmless when independent but biasing when shared; and regularization injects the prior that connectivity is sparse and sign-consistent.

Key references. S. Rotter & M. Diesmann, *Exact digital simulation of time-invariant linear systems…*, Biological Cybernetics (1999). N. Brunel, *Dynamics of sparsely connected networks of excitatory and inhibitory spiking neurons*, J. Comp. Neuroscience (2000). P. Rupprecht *et al.*, *A database and deep learning toolbox for noise-optimized, generalized spike inference from calcium imaging*, Nature Neuroscience (2021).